Async I/O & Worker Tuning Guide

Engineering Handbook · Internal · Owner: Platform Engineering

Purpose

This guide is how we write async request paths and size worker processes so one slow dependency does not take down a whole service. Two failure modes dominate: a blocking call inside an `async def` that starves the single event-loop thread (so unrelated endpoints slow down), and an outbound HTTP call with no timeout that hangs until every worker slot is consumed. The rules below keep the loop free and make every outbound call fail fast. This applies to api-service, gateway, ingestion-service, and worker-service.

Standards

Never block the event loop

An `async def` route runs on one loop thread per worker. A synchronous network call (`requests.get`), a blocking DB driver, or heavy CPU work inside that coroutine blocks the loop, so P99 latency spikes across all routes on that worker. Use an async client for I/O, and offload unavoidable blocking work to a thread.

```
# WRONG: blocks the loop
@app.get("/proxy")
async def proxy():
   return requests.get(UPSTREAM, timeout=2).json()   # sync call in async route

# RIGHT: async client
client = httpx.AsyncClient(timeout=httpx.Timeout(2.0, connect=1.0))
@app.get("/proxy")
async def proxy():
   r = await client.get(UPSTREAM)
   return r.json()

# RIGHT: offload blocking/CPU work
result = await asyncio.to_thread(cpu_heavy, payload)
```

If a handler must use a blocking library and cannot offload, make the route a plain `def` so FastAPI runs it in the threadpool instead of on the loop.

Every outbound call has an explicit timeout

A client with no timeout waits forever when a dependency is slow, pinning event-loop slots (async) or worker threads (sync) until the service stops accepting traffic. Set connect and read timeouts on every client and fail fast.

```
# httpx: separate connect vs read
timeout = httpx.Timeout(5.0, connect=2.0, read=5.0)
async with httpx.AsyncClient(timeout=timeout) as client:
```

```
    r = await client.get(url)

# per-operation deadline on top of client timeout
r = await asyncio.wait_for(client.get(url), timeout=6.0)
```

Combine timeouts with capped, jittered retries and a circuit breaker so a slow dependency degrades gracefully instead of hanging the caller. Only retry idempotent operations.

Size workers for the workload

- Use uvicorn workers for async apps: one process per CPU (`--workers $(nproc)`); each runs its own loop. Do not add threads to “fix” a blocked loop — remove the blocking call instead.
- For sync/blocking handlers behind gunicorn, use `gthread` with a bounded thread count and a request `--timeout` larger than the outbound read timeout, so the client aborts before the worker is force-killed.
- Bound outbound concurrency with an `asyncio.Semaphore` so a burst does not open unbounded connections to a downstream.
- Reuse one long-lived `AsyncClient` per process; do not create a client per request.

Common failures and fixes

- P99 spikes and unrelated endpoints slow down because an async route makes a blocking call (`requests.get` / heavy CPU) → switch to `httpx.AsyncClient` or offload with `asyncio.to_thread`; never call blocking I/O directly inside `async def` (BUG-1015).
- Requests hang and worker/loop slots are exhausted because an outbound client has no timeout → set explicit connect and read timeouts on every client, fail fast, and add retries/backoff plus a circuit breaker (BUG-1068).
- gunicorn `WORKER TIMEOUT` under load → move blocking I/O off the request path, raise `--timeout`, and add uvicorn/gthread workers for concurrency.

Checklist

- No blocking I/O or heavy CPU inside any `async def`; such work uses an async client or `asyncio.to_thread`.
- Every outbound HTTP client sets connect and read timeouts; no client is created without one.
- Retries are capped, jittered, idempotent-only, and paired with a circuit breaker.
- Outbound concurrency is bounded with a semaphore; one client instance is reused per process.
- Worker count and request timeout are tuned to the workload and larger than the outbound read timeout.
- Load test confirms one slow dependency does not stall unrelated routes.

Related incidents

BUG-1015, BUG-1068